SO, ONE MORE TIME[1]: HOW DO YOU MODERNIZE LEGACY AND (PERHAPS) MOVE TO THE CLOUD?

A PRINCIPLE-BASED APPROACH

Manuel Barbero
mbarberony@yahoo.com
+1 (917) 216-0782

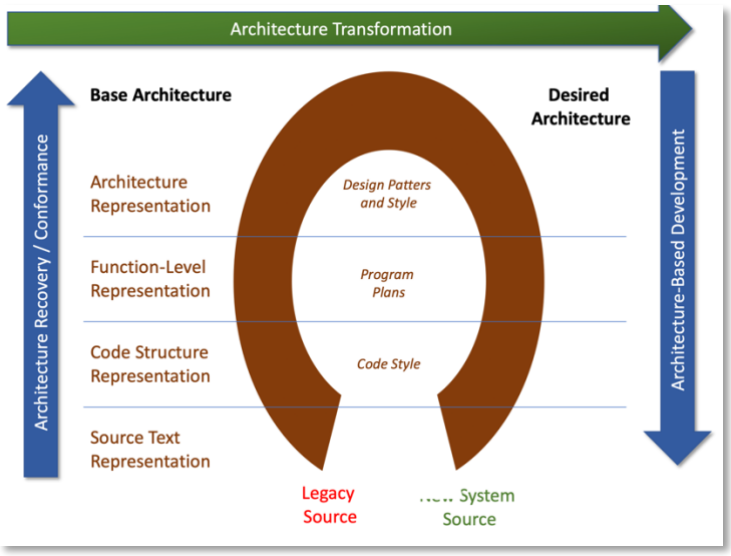
[1] With apologies to Herzberg [1].

THE CHALLENGE

These days, boards, executive management and industry analysts demand that CTOs, CIOs and Chief Architects visualize and implement solutions to key questions: "*How are we going stop spending so much effort and money in the very technology that keeps us stuck where you are?*" and this is usually coupled with "*And why aren't we moving faster to the cloud, Isn't that the way out?*"[2]

By now, porting to the cloud (at the application and infrastructure levels) as a modernization strategy is "known science" with a range of patterns well-articulated starting at least about a decade ago (see [2] and diagram, right) making generous (over-) use of the "X as a Service" trope. Software legacy modernization has sprung an even older set of approaches. One of the most popular is the model promoted by the Software Engineering Institute (the SEI "Horseshoe" model diagram below) which goes back to … 1999 [3].

This model stipulates that renovation of a given piece of software consists of two fundamental phases:



- in the ***Analysis*** phase, the code's "as is" structure/architecture is recovered and is checked for conformance with known or desired best practices/principles. The code is first parsed (from the bottom up) for structure patterns (essentially functions in the 1999 model) to recover the "base" structure/architecture;
- the ***Synthesis*** phase consists of applying (hopefully better) "to be" (desired) principles to modify the architecture at the *conceptual* level which then translates into the implementation at the *logical* (functions) and then code / *physical* levels using an "Architecture-Driven" approach to code development[3].

One goal is to identify what are the potential application architecture issues, such as remote call in loops which are typically not a problem on the mainframe and ensure that they do not become problematic in a distributed or hybrid cloud/on premises environments when latency translates into unacceptable performance[4]. This is tightly related to managing code quality standard as a whole[5] - that type of remediation can be substantially automated or systematized as we will demonstrate below.

The approach is fundamentally necessary to break monolithic applications (what is referred to as *Problem A* in *Attachment I – The two challenges in refactoring the application portfolio*), i.e. refactoring code into components that can decoupled and made into manageable pieces (e.g. role-based entitlement services) or into even bigger elements ("utilities") that can be shared across applications (e.g. a centralized payment "factory" used by all applications to interface to the rest of the world).

[2] To be sure these not only some questions tormenting CIOs and CTOs. There is also the perennial "*How can I be secure…?*", the dreaded "*What are we doing with data?*" as well as the whole spectrum of "*How can I be compliant with <Insert acronyms here like CCPA, NYPSS, …>* ?". Add to this "*How do we implement DevOps, DevSecOps, Agile…?*" (with misguided emphasis on "*Hows*" rather than asking why any of these are pertinent to the situation). The more recent question that entered the list is "*How do we leverage the 'blockchain?'"* which is joining "*How can we leverage microservices?*" in the list of buzzwords-driven musings. Although quite legitimate, I'll keep these mostly out of scope of this discussion (with an exception regarding data).

[3] For a working definition of *conceptual*, *logical* and *physical* levels (and other useful constructs) in the context of architecture see [4], section *Understanding the Difference Between the Conceptual, Logical and Physical Views…* in [4], pp. 26-27.

[4] See the attached case study *Cloud Migration – if you do only do one thing prior to shift and lift*.

[5] See "Code Quality Standards" retrieved from CISQ (Consortium for Information & Software Quality™) site https://www.it-cisq.org/standards/code-quality-standards. These are now relatively well documented in Appendix 2 particularly the tables *Architectural Practices System Level* in section *Performance Efficiency*. The Wikipedia article Software Quality at https://en.wikipedia.org/wiki/Software_quality provides an overview of the topic too.

Because it primarily focused on individual application, the narrative put together by SEI in 1999 did not put much emphasis on how to optimize the portfolio of applications as a whole. Systematizing how applications can be orchestrated along the line of business process/architecture (while rethinking the enterprise data and object model across applications) is what we call *Problem B* in *Attachment I* which provides examples of both types.

Overall, the approach needs an update to show how enterprise need to literarily visualize the overall topology of the application structure at various level of granularity, from code, to data and processes[6]. The challenge is therefore to find a way to solve *both* Problem A and Problem B – neither one of them being trivial – simultaneously in an environment where definitive answers and data is scarce and pressure to act fast is constant (See the discussion in the *Attachment II – What Is Missing From Traditional Models And Why Is It So Hard?*).

A PRINCIPLE-BASED GENERAL APPROACH TO CUTTING THROUGH THE GORDIAN KNOT

Solving simultaneous problems under uncertain constraints lends itself to using a simple but not simplistic set of known heuristics – essentially adhering to the deceivingly obvious PRINCIPLES[7]:

I. **Know what you are solving for** – Set the goal and resist deviating from it by engaging in distractions;
II. **Base decisions on the accurate description of reality and think Horizontally and Vertically –** Go "wide" when analyzing the breath of issues *and* "deep" when planning and executing;
III. **Experiment, Learn and Loop –** This is the heuristic of choice for messy/complex problems when data is scarce;
IV. **Make the Approach Sustainable** – Use *Machine Thinking*, as defined below.

The next section provides a step-by-step implementation of a recommended approach leveraging these PRINCIPLES [8]:

- **PRINCIPLE I - Know what you are solving for, set the goal and resist engaging in low-value distractions**.

  Knowing what you should be solving for is not obvious[9].

  "*Going to the cloud*", "*Implement a microservice architecture"* should not be considered ultimate goals in-and-out themselves - merely possible approaches to serve a higher purpose[10]. This said, even when non-technologists cannot articulate what a good technology strategy is, they can often "tell when they see one". So, as many companies tout themselves as de facto technology firms, it is not unfair for CIOs/CTOs to be asked to propose their goals and to demonstrate how they can be traced to their understanding of the business needs[11].

[6] At a major insurance firm we used the CAST software [illegible] viding that analysis. Some illustrations in this article were developed using that tool. There are others, see the tools section in the *Software Modernization* article in Wikipedia at https://en.wikipedia.org/wiki/Software_modernization.

[7] The list will likely come across as "motherhood and apple-pie". Yet use of these principles is uncommonly frequent. Otherwise how to explain, for example, why so many organizations implement "lift and shift" projects while this approach does not solve intrinsic issues with the application itself and perhaps is going to make things even worse through higher costs and worse performance (see attached *Case Study*).

[8] As an alumni of a leading asset management firm [illegible] largest hedge fund known for its unique culture, I am naturally biased towards believing that adhering to well-articulated principles (*Principled* [illegible] at that firm [illegible] peak) is foundational for sound problem solving. I will try to map *some* of the general Principles expounded by R[illegible] Dalio, the firm's founder [illegible] problems at hand – understanding that I probably won't give justice to the depth and breadth of his overall framework. To be clear, these references are my modest attempt to show some of these Principles at work with the caveat that my interpretation may or may not be as accurate as I hope they are. The ideas explicitly referred to in Ray Dalio's book *Principles* [5] are referred to as *Principles* (capitalized, plural) while the lower-case *principle(s)* is used in general. The four principles we recommend for the successful modernization of application are denoted PRINCIPLES.

[9] Ideally, the high-level goal for technology should be derived and traced back to the components of the business strategy (or a well-defined, transformative business initiative – defined as the most crucial component of the business strategy). But most often, these are murky, too broad or just inexistent to be effectively useful in setting the technology strategy, let alone defining an enterprise target architecture of any kind (not just technology but also business, product, process, and organizational architectures).

[10] Even *Reducing Costs* is seldom a goal that wise managers pursue. Of course, what should matter is not so much cost, but value delivered vis a vis costs (and that should definitively include opportunity costs). At any rate, computing *cost*" or *value* is at best an imprecise science. These calculations tend to of the *High precision/Low Accuracy* kind. Statistical methods like Monte Carlo simulations can help (see [6] and [7]) understanding that few technology organizations have the managerial bandwidth or the analytic skills to do this right.

[11] One way to formally trace technology goals to business objectives (if they exist) is to use House-Of-Quality-type analysis. See [8] for an example of such tracing/mapping approach albeit in a different context namely linking architectural principles to business requirements.

So, what *IS* a good goal? Here are two simple rules/tests:

A worthwhile goal should be articulated in a way that the business can value and at a level that the business can relate to. "*Create customer loyalty and thus recurring revenues by providing a seamless, superior user experience across devices and entry points into the firm*" is one example of tangible objective at the highest level (Level 1). Achieving that goal might requires solving issues at a lower level (Level 2) e.g. "*renormalizing data models*", *"implementing rule-based workflow management*" or "*moving from batch to event-driven architectures*". This, in turn, might require achieving goals at the even lower level (Level 3) through a combination of technology-specific approaches or paradigms (say, using the micro-services or object-orientation architecture styles). But starting at level 1 is a prerequisite as jumping to other levels is misguided if these subaltern goals can't be traced to a Level 1 goal. They are at any rate, meaningless to a business audience.

Another useful way at testing if a goal at <u>any</u> level is indeed worthy of attention is to ensure that it will position the firm to generally adapt and move fast as business conditions change[12]. In effect, many investments in technology, (particularly infrastructure) should be priced as *barrier options*, a way to acquire the ability, in the future, to makes yet-undefined but demonstrably valuable activities possible [9]. Indeed, this is fundamentally why legacy technology is an issue: it's a burden that needs to be carried forward and slows the enterprise down by reducing its ability to adapt. In the previous paragraph, the goal of introducing rule-based, event-driven workflows is easily traceable to being able to introduce new products/services in a way that will keep the user experience seamless and rewarding.

- **PRINCIPLE IIA – Base decisions on the accurate description of reality – Know when to go wide to analyze the breath of issues** – Once the goal is set (i.e. what do we fundamentally need to do to be ready to adapt), knowing what to tackle and in which order is critical.

  A *faux-truism*[13] is that planning should be fact-based[14]. It demands commitment to radical transparency and open-mindedness to systematically looking at instances when things went wrong against a well-defined goal, then specifically identifying what prevents you from achieving the goal, before looking at the various possible options[15] and opting for the optimal one, i.e. the one most likely to address the root cause(s) of the failures[16].

  The quest for understanding what you have to deal with *<u>at the aggregate</u>* and what to do about it is probably the single biggest issue facing technologists when they attempt to scope out their plan and this is a step currently missing from the SEI Horseshoe model.

  From a practical standpoint that means making sure that you pick and choose which application to target first (and why) based on a fact-based analysis. It also means setting up the right metrics to track how your trajectory stays within a valid *cone of expectation*[17]. Both require the analytical framework presented next.

[12] In fact, one can define the goal and the role of the architecture function as insuring that "the right technology decisions are made". This is what *Architecture* as a discipline is all about. Establishing technology governance, setting standards, visualizing target state architecture and roadmaps, and ensuring that the right remedial actions are taken when implementation of these plan are not going as planned, this are merely activities necessary to make informed decisions based on *sound principles* and *Truth (as defined in footnote 14)*.

[13] A truism is defined as *a recommendation that on the surface is obviously true and says nothing new or interesting and yet not actually implemented as often as it should*. Let's define *faux-truism* as a recommendation that on the surface might come across as a truism but that in fact is much more insightful that it appears and quite difficult, in fact, to adhere to. Many (if not all) Principles qualify as faux-truisms.

[14] This is at its core what Dalio promotes in general, i.e. *Trust in Truth*, *Truth* being defined as the *accurate description of reality,* see [5] "1. Embrace Reality and Deal with It", p 135 – This seems to be the cornerstone of his thinking. For technologists this means staying away from and pushing back on pressure to either a) rely undiscerningly on whatever technology problem-solving mantra *du jour* or b) skip defining good goals before jumping into conclusions without going through a systematic analysis (see footnote 16 on the *Five Step Process*).

[15] This notion of explicitly tracing option selection to requirements, themselves linked to foundation principles/goals is not new either: as noted in footnote 11, it's the essence of using Design for Six Sigma's House of Quality analysis, something that is surprisingly rarely used by technologists. For an example, see [8].

[16] The sequence of actions described here is closely related to the *Five Step Process* in [5] "Chapter 2. Use the 5-Step Process to Get What You Want Out of Life", p. 168. There is an important distinction in [5] between *root* and *proximate* causes, see footnote 27.

[17] A *Cone of Expectation* (for a project) is de facto defined in [5] as the cost/value data range between the best- and worst-case scenarios plotted over time. The "cone" terminology reflects that uncertainty makes the two worse/best case functions diverge inexorably. A good way to track progress effectively is to ensure that it stays within that code of expectation rather than to react to normal, literary insignificant fluctuations over time. In other words, what matters is trajectory, not just point in time values as expressed in [5] "5.3.a – Keep in mind both the rate of change and the levels of things and the relationships between them", p. 244.

- **PRINCIPLE IIB – Base decisions on the accurate description of reality - Know when to go deep when planning and executing.** The SEI model posits that in order to refactor code, one needs to systematically extract its structure from the textual source, then the functional and the architectural representations of the application under modernization.

  Notwithstanding that critical insight, there are two dimensions that the original model does not explicitly refer to and that we will add to recommended approach:

  - **Data** – While understanding how business rules are embedded in the code is key[18], harvesting what data model the original designer(s) used to represent the world (aka, hopefully, *reality*) is equally important, in fact doing so systematically is nowadays an imperative given the regulatory and compliances demands. Regulators want companies to demonstrate that they know where data is. how it is processed, and how it can be expunged from the records per customer requests[19].

  - **Cross-Application Dependencies** – Anyone tasked with trying to gain an understanding of legacy code comes to quickly realize that a lot of complexity resides in the (at times infuriating) level of mutual dependencies that has been be built over time by harried coders under pressure to quickly connect distant code units. By definition, understanding and re-organizing cross-application dependency way to solve Problem B[20].

  To be clear, this is all about identifying structure *not* dynamic behavior[21] and best done through automated static pattern analysis and documentation. This must be performed with an eye on identifying opportunities to refactor application functionalities (and restructure the enterprise data plane across the application portfolio).

  What is needed are tools that analyze the legacy code and *understand*[22] what it is actually *doing*, i.e. how transactions are built up and executed, how is data accessed, what are the explicit relationships between components to visualize the outcome in a way that can be exploited by architects in charge of the application portfolio transformation.

  The alternative is to task architects to document the system's inner structure by hand and often based on tribal knowledge, but that's slow and more costly than automation that can analyze code at the aggregate (and yet in depth) to literarily visualize and inspect code and data structures at various level of granularity.

[18] There are also an emerging crop of tools to harvest business rules, mostly provided by traditional outsourcing vendors as a component of their service offerings – They are mostly editing aids / IDE-like tools that help a) codify language-specific indirection-based logic (if … then else …) into a common syntax and b) refactor variable names so that the result can be used to analyze and then re-code the logic either in a new target language or as rules in a rule engine. Although these tools accelerate rule discovery and documentation, they still are highly dependent on "human" insight. Perhaps AI/Natural Language Processing will help increase the level of automation in the future.

[19] This is a huge opportunity to help answer some of the key questions mentioned in footnote 2: "*What are we doing with data*?". Lack of space prevents us to go into more details but suffice to say that harvesting data structures as they exist in the application portfolio is probably one of the best ways to start refactoring the enterprise data model. Understanding (and refactoring how) each application represents business objects makes it easier to organize data into well-formed domains – one no longer tightly connected to individual applications, perhaps going away from the tyranny of having to establish "system(s) of record", a laudable but possibly (for reasons too long to explain here) ill-advised goal.

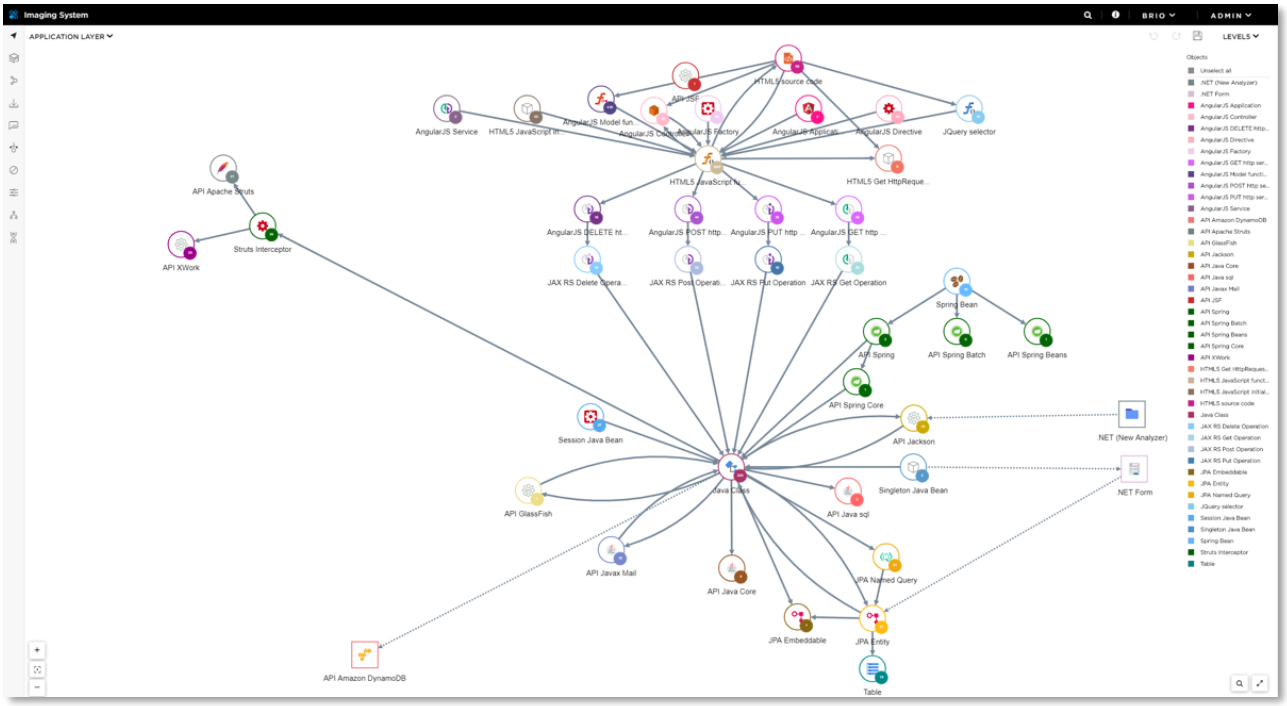
[20] *Spaghetti code* and, worse, the topology of *spaghetti systems* both introduce convoluted data exchanges, Rube-Goldberg arrangements that create extreme latency in processing information and intractable testing challenges. It also makes any kind of code (or even infrastructure) changes a risky proposition. There have been reports of how modifying a footnote in a report brought down a system in a Fortune 100 company. No wonder that introducing new features in that kind of environment can only be done at a glacial pace.

[21] There are many tools such as Device42, Dynatrace, AppDynamics, Movere, Cloudamize or Turbonomic, that provide application discovery, infrastructure assessment, and app to app dependency analysis. But they are doing so by monitoring the performance of applications at runtime and/or in production. This is often referred to as Application Performance Monitoring (APM) or Dynamic Application Analysis. For a survey of such tools, with a focus on use for cloud migration see [10].

[22] Defined as *the process of automatically analyzing artifacts of a software application and determining metadata structures associated with the application in the form of lists of data elements and business rules* in https://en.wikipedia.org/wiki/Application_discovery_and_understanding.

In fact, navigating the levels of code coarseness is critical to form a high level and then increasingly refined view of what needs to be dealt with[23].

The diagram below[24] shows an example of such a view. In this simple case, the architect has identified a component of the architecture (use of obsolete STRUTS framework in the upper left corner) as a candidate for replacement.



- **PRINCIPLE III - Experiment, Learn and Loop -** That solving complex problems (such as organizing the rewrite of a portfolio of legacy apps) should be an iterative process is now a trope widely understood by technologists. It is for example, at its core a foundational element of agile thinking.

  Yet without incremental leverage of lessons learned, looping is merely churning. Many organizations tackle legacy application remediation as individual projects. One of the reasons is lack or cross-portfolio governance (see next PRINCIPLE) or just the operational difficulty of having several teams making time to harvest, much less share, lessons learned in their individual attempts to remedy codes.

  But at its core, this missed opportunity stems from the simple fact that organizations – in an attempt to "stay focused" – do not recognize harvesting and leveraging lessons learned across projects as a worthwhile goal.

  This is something that keeps organizations on a low growth trajectory. Sure, the "retrospective" step in agile development is an instantiation of such thinking at play but in practice, the systematically harvesting and sharing knowledge gained from refactoring is rarely promoted across the organization[25].

  Minimally, an understanding of how each application models data and business objects should be part of what each team should document so that the overall enterprise data ontology can be shaped, and data representation be rationalized in, for example, well structure data domains. This is why this step is explicitly introduced in the recommended new approach (next section).

[23] See [5] "5.3.b – "Be Imprecise. Understand the concept of 'by-and-large' and approximations.", p.244 and "5.4.b – "Remember that decisions need to be made at the appropriate level, but should be consistent across levels.", p. 247. Although both of these principles are general in nature, they also remarkably apply to how to look at code analysis as well.

[24] The chart and the next one are based on the use of CAST Highlight tool see https://www.castsoftware.com/products/highlight

[25] This *feedback loop* is an essential component of the *Machine* model described in the next, closely related PRINCIPLE.

- **PRINCIPLE IV - Make it sustainable: Machine Thinking -** When it comes to application modernization (or remediation) many technology organizations fail to set-up processes that can be used over and over. This is a failure to apply *Machine Thinking*.

  Machine Thinking[26] is a way to operate well and deliver good results by insuring that

  - a *goal* is well defined to deliver a specific *outcome*;
  - a person (*Responsible Party* or *RP*) is responsible to achieve it;
  - the adequate group of people are selected to operate a series of well-defined processes (collectively, the machine's *design*); and crucially,
  - the RP is under the obligation to constantly look at the machine's outcome and tweak the overall machine's design (people, processes) to improve the trajectory of execution vis-a-vis the desired outcome.

  Machine Thinking seems obvious, but this is another faux-truism: the reader might ask themselves how many times one or more of the characteristics above are missing from their engagements with others. The reality is that unclear goals, dilution of responsibilities, inadequate staffing (size and abilities/skills) and half-backed or undefined processes are rampant in most organizations. We also noted above how rarely organizations implement or value the need to develop systematic, continuous improvement[27].

SO, ONE MORE TIME: HOW DO YOU MODERNIZE LEGACY AND (PERHAPS) MOVE TO THE CLOUD? A STEP-BY-STEP APPROACH

Machine Thinking requires that we close the loop on the SEI horseshoe model (and probably find a new name – we will call it the *Modified SEI (Horseshoe) Model*).

After the all-important steps of setting both expectations (Step 0) and the support infrastructure (Step 1 - Tools, people and process) and a first cut at prioritizing the portfolio (Step 2), the approach consists in a series of loops (Step 3) that builds on the SEI model.

Step 0 – Set Expectations

- **Communicate the goal and approach with stakeholders and get in hard sync with them[28]** – The expectations should be that this will be a *journey*. And that the journey will have both an *execution* dimension (where results will be achieved) as well as an – intermingled – *on-going learning and refinement* dimension (to constantly improve a *Machinery*).

- **Size-up the opportunity at the high level** – Compile a broad outline of what can be said about the overall size of the effort and therefore its duration (using T-shirt sizes for both and ideally, Monte Carlo analysis to define confidence ranges [7]). But be ready to push back on demand for a traditional "plan". This is a case when, by design, the plan consists in setting a direction and looping through an iterative process which itself will evolve over time.

[26] The concept of Machine is one of the central one described in Dalio's Principles. See *Principles* "10 – Manage as someone operating a machine to achieve a goal", p. 448. A group of related machines is referred to as a *Machinery* (in which one of the Machine should insure cross-interaction among the group as well as on-going refinement of the Machinery itself!)

[27] See [5] "10.2 – Remember that for every case you deal with, your approach should have two purposes: a) move you closer to the goal and b) to train and test your machine (i.e. your people and your design", p. 453. Many technology projects fail to implement b) in general and application modernization is no exception. Even when teams go through "retrospectives" (in the context of agile development) they tend to deal with "proximate" rather than "root" causes (the ones that relate to people values, capabilities or skills). Effective "diagnosis" of issues is central to Principle Thinking and a key component to the general "5 Step Approach" to problem solving – See [5] "Chapter 12 – Diagnose Problems to Get at Their Root Cause", p. 483.

[28] See [5] "4.1.a – Spend lavishly on the time and energy you devote to getting in sync, because it's the best investment you can make", p. 360.

- **Resist the pressure to provide a precise cost/benefit analysis prematurely.** Rest assured (and be confident you can tell management with high/medium confidence) that the proposed approach is designed to minimize the initial investments (in setting a small team to operate the proposed approach/machinery as a Proof of Concept and gathering high level data) will be modest, leading to an informed go/no go decision before proceeding further.

To be clear, management should rest assured that costs and benefits WILL be monitored as part of the project machinery. No CxO will (or should) move forward and support an initiative (even at the proof-of-concept level) without a clear narrative as to what the overall approach is. But rather than opining (or not) to a series of numbers, the best CxOs probe their subordinates on the soundness of the model they use, asking what data was fed into the model at first and how both data and models will be made to evolve over time.

Our recommendation indeed IS to get ready to answer these questions (using this article if it makes sense) while avoiding the *high precision / low accuracy* pitfall.

Step 1: Organize the Team and the "Machinery".

- **Set up a small but well-structured team with participants in hard sync with the goal and the approach**. As mentioned above, starting small is more likely to succeed because it will enable the team to prototype machines (see list below) and refine them over time[29].

- **Set up The Machinery** - In addition to running the actual steps of the modified horseshoe model, we will need a few key machines to support key goals:

    - **Ensure that the project stays on track:** Constantly measure progress and how the trajectory fits into the right cone of expectations over time. This is where having selected the right metrics[30] and gathered the initial data in the previous step is leveraged;

    - **Ensure that you are and stay in touch with stakeholders:** This is about providing radical transparency on what is done and where you are headed (vis-à-vis the trajectory in the cone of expectation)[31];

    - **Last but not least, ensure that the overall machinery improves over time:** through real time constantly reassessing how to improve or modify the overall machinery[32].

Step 2 – Scan the Portfolio and Prioritize.

This is about harvesting the right information to make rational decisions on what to prioritize. There is a simple set of those that tend to be always relevant and tools to support this process as described below. One key dimension to consider must probably be "Readiness to Cloud Migration"[33] which means knowing two types of information:

- **Size-related information** (basically size/complexity of the code base and footprint of the supporting infrastructure);

[29] To be clear, this is not about immediately setting up a typical PMO structure although there are overlapping characteristics. PMOs tend to induce *High Ceremony* overhead (see [4], p. 23). Of course, the transformation might evolve towards using a factory-like series of activities that require some heavy-duty processes and a full-fledge PMO. But the core team operating the three machines listed in this section should stay modest in size while the work is performed by engineering teams (using say, agile methodologies).

[30] Selection of specific data and metrics is discussed immediately below in the description of Step 2.

[31] And be ready to be challenged, in fact invite and welcome dissent – [5] has an entire chapter about decision making weighting believability (pp. 370-392). One of the many Principles that applies here is "5.2. Find the most believable people possible who disagree with you and understand their reasoning", p. 375.

[32] See [5] "10.1 – Look down on your machine and yourself within it from the higher level", p. 450.

[33] Ideally the dimensions should be mapped to the overall modernization project's goal defined in Step 0, perhaps through a *Quality Function Deployment* analysis described in [8].

- **Risk/Reward-related information** (typically, understanding the business impact of the application, its readiness to be ported to the cloud and opportunities to refactor application components across several applications of the portfolio).

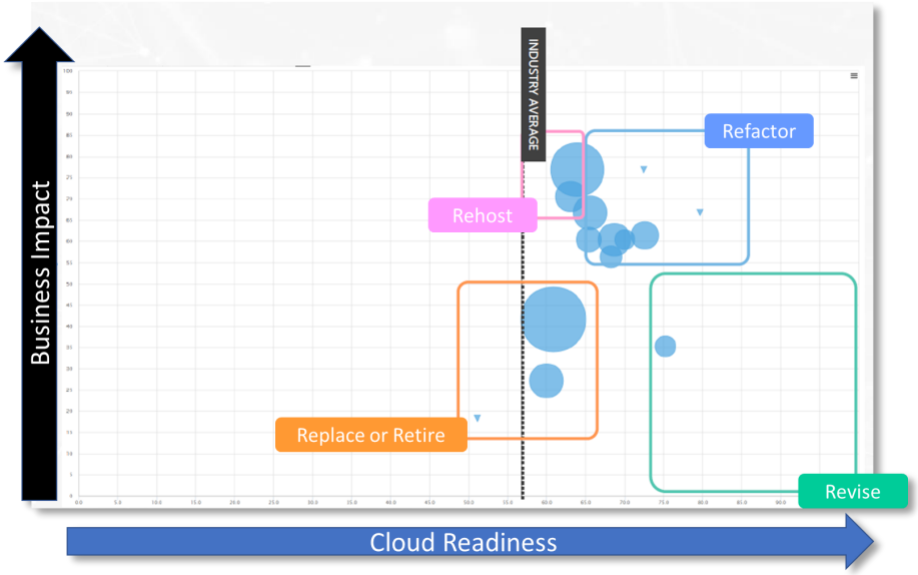
Most organizations have *some* initial data regarding these matters perhaps as a by-product of (for example) the Disaster Recovery Planning efforts that demand that key application's metadata be documented, or as a result of other risk or regulatory mandates or even financial accounting activities (e.g. license inventory compliance).

But because these efforts had intrinsically different goals, the data gathered are unlikely to be usable "as is". So, there are no short cuts: the technology group is likely to have to build the right data set to scope and prioritize out the modernization effort.

In many instances, technology organization need to rely on surveys: gathering relevant data is about asking the right questions and here again, one need to resist boiling the ocean and, fortunately, there is a simple set of questions that in practice help cut through the analysis including (this is not an exhaustive list)[34] :

Category	Questions	Purpose
Business Impact	Does the application serve internal or external users?	Identify potential impacts to customers if there was an outage
Business Impact	Could failure of this application lead to loss of revenue?	Identify financial impact if there was an outage
Cloud Ready	Is the application multi-tenant?	Estimate ease of migration
Cloud Ready	What is the average skill level on cloud technologies of the development team?	Estimate timeline for migration
Cloud Ready	What is the current user authentication mechanism?	Identify security implications before migration
Application Properties	What is the type of application?	Identify potential redundancies or consolidation opportunities across the portfolio

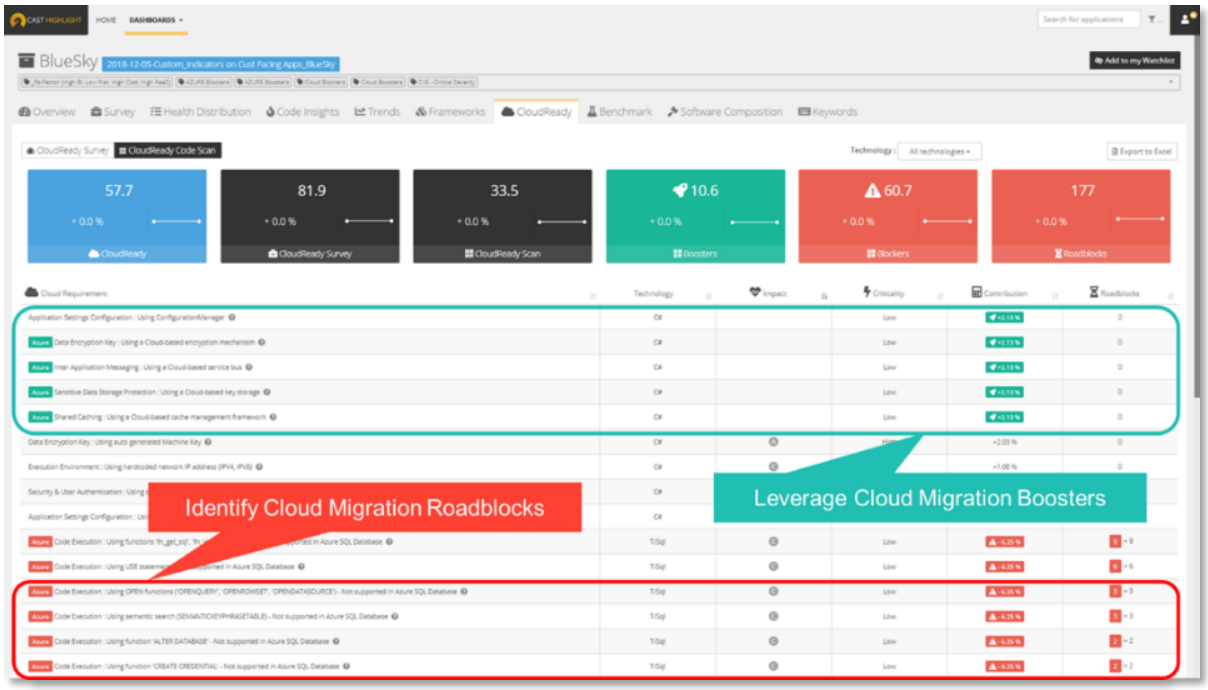
There are several tools that can help to perform an initial analysis of the portfolio[35], set up metrics and thus assessing best candidates. The tools leverage these as metrics identify quick wins based on a how these questions are answered. The breath-first, *by-and-large*[36] inventory of custom application will provide a broad sense of how to proceed. The diagram below shows an example of how a roadmap to the cloud for large application portfolios can be developed using both objective cloud readiness measurements and qualitative business context considerations.



[34] See https://www.castsoftware.com/products/highlight/cloud-readiness-paas-migration
[35] See list of tools and a discussion here https://en.wikipedia.org/wiki/Application_portfolio_management and https://www.gartner.com/en/information-technology/glossary/application-portfolio-analysis.
[36] See footnote 25.

Each bubble in this chart represents an application in the portfolio. Assessing each application by both objective data (from a code scan) and qualitative data (from surveys) enables an enterprise to segment and prioritize applications more intelligently into categories such as Refactor, Revise, Rehost, Replace, and Retire defined in [2].

The scanning process can also identify specific cloud migration *Blockers* and *Boosters* within the source code of an application that is slated for migration to the cloud. Boosters are good practices that make an application more cloud ready (e.g., using an existing cloud-based encryption mechanism). Blockers are hurdles that will prevent cloud migration or make it much harder if not remediated (e.g., using persistent files).



Step 3 – Run the steps of a modified version of the SEI model presented earlier in this article.

There are essentially four core modifications to the model (see diagram below):

A – **Have each loop start with a reassessment of priorities** along key dimensions (**PRINCIPLE I** and **IIA),** refining what was done on Step 2 at each round;

B – **Put emphasis on identifying not just *execution* logic but also harvest *business logic* and *data structure*** information (ideally in an object-oriented model) (**PRINCIPLE IIB);**

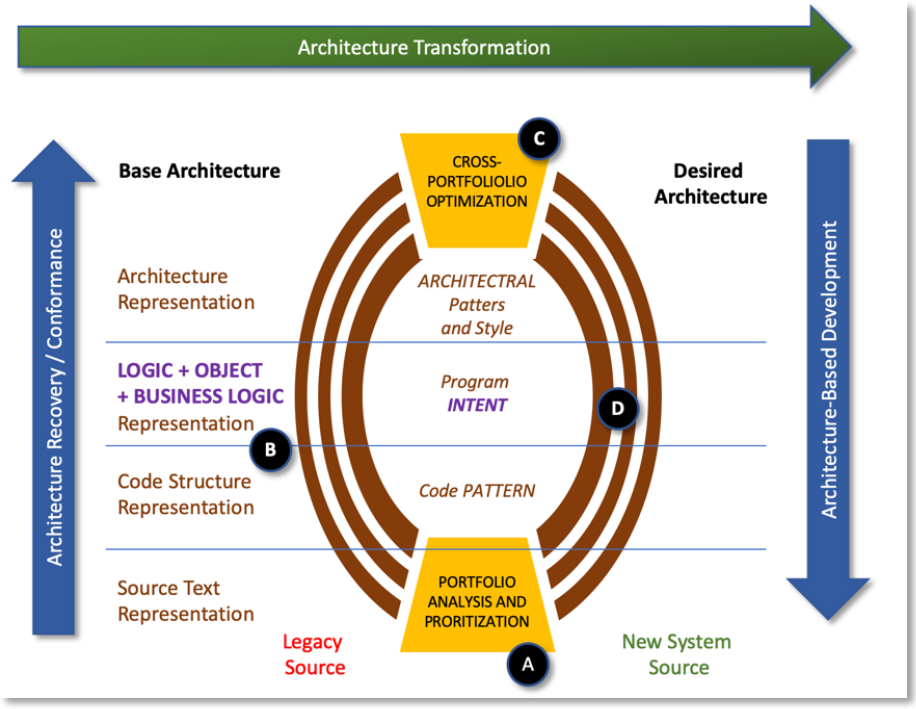
C – **Constantly look for optimizing the portfolio by:**

- ***Refactoring the applications in modules*** that can be shared across applications; and,
- ***Implementing cross application orchestration*** to support better architected business processes.

This is where we can leverage the significant body of work that has been developed by industry groups for best practices in:

- Performance/Efficiency [11],
- Security [12];
- Code quality in general[37]

[37] See https://www.it-cisq.org/standards/code-quality-standards/ already mentioned as reference in footnote 5.



D – "Rinse and Repeat" the process for a new select group of applications. This can be executed as a parallel process targeting a variety of applications (**PRINCIPLE IV)** and building knowledge to fine-tune the overall process over time (**PRINCIPLE III** and **IV**).

Observe that the new model calls for the parallel execution of the modernization loop across *several* applications. It does not mean however that the modernization should be attempted at the same level of depth. In fact, when refactoring a portfolio of applications, it is best to go through three stages:

- A ***Proof of Concept*** stage that tests if a given target conceptual architecture (across applications or for a given application) can be implemented, which each new component mocked-up (for example in the commission system example mentioned as Problem A in the *Attachment I*, one would prove that a rule engine could be included in the design although not all business rules should be implemented);

- A ***Pilot Phase*** stage that delivers a Minimal Viable Product that can be used to manage a subset of the problem at hand (For example, in the example in *Attachment I*, implementing the Proof of Concept for a limited number of agents focusing on a subset of the whole product portfolio but at scale with all the accoutrements of an application in production). That phase will provide tangible information to size-up the effort for the next phase;

- A ***Scaling Phase*** that builds on lessons learned from the previous phases to get to a solution that can be operated as a full-fledged substitute of the original application (All users, all functionalities).

A first round of execution could start with just one Proof of Concept, then adding another one as the first application goes to a Pilot Phase etc. so that at any given time the team manages a portfolio of applications at various stage of transformation,

This is essentially about *starting small and iterating* – a way to deal with the dearth of reliable information we identified as a key challenge at the onset of the discussion, dealing with the issues described in *Attachment II*.

SO, ONE MORE TIME - HOW DO TO MODERNIZE LEGACY AND MOVE TO THE CLOUD?

The goal of this document was to help technology leaders answer key questions and explain what is missing from traditional model**:**

- **Q: How do we get started**?

  A: By ensuring that expectations are set correctly, that change is organized as an iterative process, and that risks are managed in a transparent manner that allows for on-going improvements. Starting with a Proof of Concept of the approach and moving on to Pilots for a small group of application specifically selected for their potential is the way to start.

- **Q: How do we know we are targeting the right goals and that we are on the right trajectory to get there?**

  A: The approach calls for a periodic and explicit measurement of achievements against fact-based information. This can be done by scanning the portfolio both in depth and in breath to guide priorities. Selecting which applications should be modernized (and/or ported to the cloud) will be based on known rational criteria that will also be used to track the trajectory of the overall project. All this is can and will be aided by tools which makes it realistic to manage the trajectory of such projects in a systematic, data-driven way.

- **Q: When will this be done? How much will it cost?**

  A: Those are questions that should only answered at the by-and-large level first and refined over time. The alternative is to commit to a flawed and unrealistic plan. Initial investment should be modest at first because a big-bang approach usually fails.

Our hope is that the discussion will not only guide practitioners in achieving their goal but also disseminate common-sense ideas that executive management can attach to regardless of their technology proficiency.

And what common sense has taught us is that in an environment where definitive answers and data is scarce and pressure to act fast is relentless, setting up relevant goals and set fact-based expectations is key. That implies avoiding jumping to quick conclusions and constantly adjusting estimates and approach along the way by setting up the overall project as an evolving machinery.

In addition, nowadays and fortunately, tools that can really help remove the tediousness of gathering and synthesizing key information about the enterprise software (both in terms of breath of portfolio and dependencies) and deep into the code up all the way to the overall ecosystem architecture.

Leveraging these tools combined with the right (principle-based) execution practices can guide CIOs and CTOs throughout the marathon that is legacy application modernization and moving some of them to the cloud.

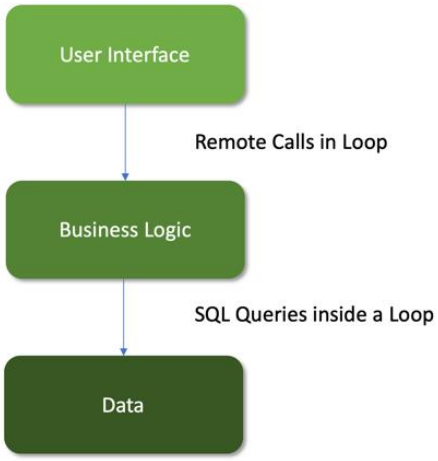
***** *****

About the Author – Manuel Barbero has been involved in the business of managing technology for over twenty-five years. He has held CTO, Head of Development and Chief Architects positions at major financial institutions, including a leading insurance firm, a global hedge fund, and UBS. He also run the financial services technology advisory *practices of KPMG Consulting / BearingPoint and is involved in the management of several start-ups in a range of industries. He is a geek, table tennis, math enthusiast with three (perfectly legal) passport, and a Computer Science MSc engineering degree from Telecom ParisTech. His MBA and degree in Artificial Intelligence are from Columbia University.*

CASE STUDY – CLOUD MIGRATION – IF YOU DO ONLY ONE THING PRIOR TO *SHIFT AND LIFT*

Consider the following application (illustrated at left), typical of today's architecture:

- A front-end made of HTML5/JavaScript/modern frameworks accessing a back-end of services through REST calls composed of two additional layers;
- A business logic layer developed for example in Java;
- The data access layer in Java tool leveraging a persistence framework and the data services proper, for example, a regular RDBMS and a NoSQL platform for the unstructured data.



In a classic on-site implementation, all the components would be running on the same network using a properly dimensioned compute and storage infrastructure built with this assumption in mind (i.e. accesses to data is fast and cheap).

In this environment, is not uncommon that the communication between the layers, specifically from the business logic to the data storage layer to not be optimized and/or reduced to the minimum. Developers may have even leveraged the opportunity of fast data accesses and/but then (alas) adopt non optimized patterns such as looping calls from the business logic layers to the data storage layers instead of writing a single optimized query to gather all the data in one call.

Once moved on a cloud infrastructure, the assumption at the core of the architectural and development choices will no longer be valid anymore as one has little control on how the different services will be deployed by the cloud service provider. For example, by default, the performances of the exchanges between the different virtual machines is constrained by a range of implementation consideration including cost.

One remedy would be to implement a dedicated set of machines with a significant guaranteed bandwidth in between the machines, reproducing the configuration that was on premises, but that would defeat the original intent of moving to the cloud (cost reduction, dynamic resource allocation…).

The situation can be made worse if, to preserve data confidentiality or to satisfy other compliance constraints, the data is persisted on premise. Then the hundreds of exchanges between the business logic and the data storage layer that were easily absorbed by the on-premise infrastructure become a bottleneck and eventually impacts performances. What makes this insidious is that these issues can crop-out over time unnoticed at first but quite perplexing over time.

And even in the absence of loops in the design and despite the use of persistence frameworks, things can *still* go wrong: it took once a team of subcontractors about a week to identify that caching parameters were wrongly set in the Hibernate framework they used – This goes to show that systematic review of code delivered by third parties should go beyond just detecting conformance to security-related coding practices. And that regardless of how good your teams are, you can't assume that your vendor implementation team is intimately aware of the innards of the solution they help deploy.

The smart move here is to identify all potential issues in the application before the actual move. There are various approaches to eradicate the issue. That includes re-performing unit test to profile where bottlenecks are. Better yet, use automated code analysis technology described in this article to spot engineering anti-patterns across multi-tier architecture, one that can also scan through persistence frameworks[38].

*** ***

[38] See footnote 6 about https://en.wikipedia.org/wiki/Software_modernization

APPENDIX I - THE TWO CHALLENGES IN REFACTORING THE APPLICATION PORTFOLIO

Organizations place an intense focus on individual legacy application modernization (we will call this *Problem A.*

Take, for instance, a commission system used by financial services organizations to manage the variety of compensation schemes used to incentivize their distribution channels (independent or captive agents).

For example, service ***S*** sold by agent ***A*** will result in a one-time commission ***C(P,A)***, a function of ***S*** and ***A***. But things can be vastly more complex, for example in the life insurance industry, a Policy ***P*** was sold ***n*** years ago under discount structure ***D*** by an agent will translate into a recurrent commission ***C(P,A,n,D)*** to ***A***. And the rules become even more complex for team sales and other promotions schemes introduced over time.

Ideally, these systems should consist in:

- A ***data ingestion and normalization layer*** that captures the relevant information required to compute rewards, ideally from various systems of record for a) service-specific characteristics / pricing and b) distribution agent- specific attributes;
- A ***rule engine*** that applies the necessary algorithms like the formula above;
  A ***synthetization layer*** that aggreges results at the agent level (to be published say, every month);
- A ***general cross-system workflow and exception management*** sub-system.

In practices, legacy systems evolve in such a way that these layers are seldom so neatly decoupled or even logically organized[39]. In addition, rules are hard-coded in a series of lengthy "if..then,.. clauses" that are very risky to change over time. The whole thing becomes very brittle[40] and soon, no new product can be introduced, endangering the long-term relevance of the company. This is a typical Problem A.

But even if these monolithic systems are re-factored, one need to give a lot of attention to how to re-stich together these applications in a way that create velocity and economies of scale and scope at the *enterprise* level.

If one merely decouples the components of monolithic apps without recoupling refactored components into re-usable, components that can be mutualized across the firm, one fails to leverage an opportunity to truly digitize the organization at the business process level.

This is not just a “nice to have” – It is difficult if not impossible to create integrated business processes if they are impeded by the fragmentation of the underlying application portfolio. That translates into disjointed user experiences (for example having to re-enter data at the home office or in the field when jumping from one application-specific screen to another).

And it is also a lost opportunity to redesign so-called *User Journey(s)* (or the home office workers' daily experience) and make them smooth across platforms at each contact point ("Moment of Truth"). In fact, not rethinking how the data flows and the underlying data models can be recomposed is not just a huge missed opportunity but a *guarantee* that the actual fragmentation of user experience will continue. It makes a key worthwhile goal (as defined in the *what is a good goal*? discussion in this paper) impossible.

This is *Problem B*: *how to modernize the application and integrate then to implement actual intelligent, integrated workflows and introduce economies of scale and scope*. The only way to achieve a worthy goal like *Creating customer loyalty and thus recurring revenues by providing a seamless, superior user experience across devices and entry points into the firm* cited earlier in this article.

[39] For historical reasons that *might* have made sense at the time they were made, some systems would go through a systematic calculation of *all* possible commissions for all agents and *then* screen out the results for those agents who did not qualify for them. That’s one way of doing it, if for example one relies on batch to process large amount of data when storage costs were at a premium, probably not the best under other circumstances.

[40] See anecdote in footnote 21 about the brittleness of convoluted applications.

For example, take an organization which already manages a multitude of CRM systems, say Microsoft Dynamics which might have been painfully deployed in the field over a lengthy period of time to replace a legacy Siebel system. Meanwhile Marketing might have been dabbling with Salesforce to support their campaigns. A previous version of Salesforce might have been deployed by the Call Centers to manage user requests.

And to top it off, none of them might be using the same access controls or one developed elsewhere in the organization.

Familiar?

Predictably, in this situation, voice will raise to *consolidate it all* in one single system.

Yet doing so is unlikely to yield significant benefits, at least not for a long time.

But this is actually an ***opportunity***.

In this case, the right solution might be to rethink how each platform activities can be coupled through an orchestration and data sharing layer while a common role-based access control system is implemented.

But to be clear, implementing such an approach is tricky too: some organizations try to create a full-fledged (*enterprise*) busses and *enterprise-wide* orchestration engines to connect all possible applications. Some create vast shared data repositories. Like promoting an all-API architecture to glue the ecosystem of applications, each of these efforts tend to collapse on their own weight if they are considered as a goal in-and-out themselves[41].

Given all the above and without getting into more details, solving problem A and B seems to be best done by:

- Introducing an explicit ***orchestrating interaction layer*** across apps (or their newly refactored versions);
- Using ***event-driven architecture***, implement the right patterns to exchange data across apps such as the ***lambda architecture*** or data streaming data architectures, generally, good *Level 2* goals;
- Careful using processes-aligned, ***rule-based engines***, much more flexible than imbedded code-base rules.

This approach also has the advantage of lending itself to incremental implementation through the Proof of Concept / Pilot / Scaling paradigm promoted in the Modified SEI Model.

It also helps rethinking the enterprise data model and identify the minimal necessary subset of data that can and should be mutualized across applications in a realist, incremental way.

***** *****

[41] It is true that the existence of Problem B is related to the persistence (for example) of batch processes as a somewhat primitive approach to implement orchestration. But substituting batch by having apps communicate through APIs merely address *some* aspects of the problem. It indeed improves real time performance and efficiency at the development level (code reusability, decoupling of technology components). Yet, the stateless nature of the API paradigm does not really help creating true, rule-based orchestration across applications. Performance is also an issue as APIs calls must pass context data, which introduces latency (in addition to the type of aforementioned spaghetti architecture.

APPENDIX II – WHAT IS MISSING FROM THE TRADITIONAL MODELS AND WHY IS IT SO HARD?

"As is", the Gartner and the SEI model/methodologies defined at the onset of this article are helpful in proposing execution choices and approaches but there are other, residual, practical questions facing CTOs and ICOs:

- How do we get started?
- How do we know we are targeting the right goal and that we are on the right trajectory to get there?
- When will this be done? How much will it cost?

Fundamentally, the challenge is the dearth of actionable information. This makes the questions above challenging to settle conclusively and creates a lot of unnecessary churn:

- At the highest level, many technology practitioners spend massive amount of time with limited success in trying to answer questions like *what are our critical apps?*, *what is the marginal cost of maintaining each app?* or even, at a higher level *what are the business priorities and how do I map them into my technology agenda?* – all in an attempt to apply a rigorous approach to decision making. The reality of that trying to answer these questions too precisely is, as discussed in this article, probably impossible to do but, thankfully, not necessary;
- In the technology realm, many organizations attempt to refactor applications – all at once. The issue is even more complex when the teams attempt to rationalize each application data model along the way. Again, probably not a realistic goal but in fact not a good goal because it leads to risk analysis paralysis;
- Generally, putting a price tag on the overall endeavor even for the 1 to 3 year horizon can be an exercise in futility (although many vendors will be happy to perform analysis in a few weeks – usually only on a subset of apps – the ones they know or already operate on an outsourcing basis) leading to many *high-precision, low accuracy* calculations.

Also, if you think this is hard, you are not alone. Here are the sobering numbers, collected from several sources: there are now (2019) about half a billion worldwide applications slated for modernization[42] with *Lift and Shift* being the prominent approach but with a success rate less than 40%[43]. The same data source states that the transformation journey will take more than a year and 43% more than two years[44]. In that study, 62% of people interviewed said that the projects were harder than expected or that they failed, and 55% said that they went over budget. And it's not surprising:

- Building new apps using a new *modern* stack (on the cloud or on-promises but with internet-derived technologies (e.g. *containers*, *service-oriented* architectures) and practices (e.g. DevOps)) will not obviate the need to accept the legacy technology as a *massive boundary condition*. One needs to accept that the overall technology landscape cannot be truly rationalized. For example, data residing on the legacy platforms will have to be sourced *as is* and not rethought in the context on a better, rationalized cross-application enterprise data architecture and governance model.
- When pushed to go to the cloud, technologists try to *check the box* by doing a lot of Lift and Shift, something that rarely cuts cost (compute and storage unit costs are NOT always competitive on the cloud when compared to those achieved on premises by large organization). Lift and Shift certainly does NOT provide new business capabilities. In addition, it not only creates new transition costs but also opportunity costs by monopolizing the few key people knowledgeable about legacy systems whose insight could be leveraged (while they are still around) to create better solutions.

There's got to be a better way, one which, as proposed in this article, is principled-based and supports a portfolio-wide yet iterative road to success.

*** **

[42] IDC, private communication.
[43] See [13], p. 15.
[44] See https://www.techrepublic.com/article/73-of-cloud-migrations-take-a-year-or-longer-report-says

LIST OF PUBLICATIONS AND REFERENCES

Publications and Notes

[1] – Hertzberg, F. (2003) *One more time – how do you motivate employees*. In Harvard Business Review, January 2003. Available by subscription at https://hbr.org/2003/01/one-more-time-how-do-you-motivate-employees

[2] – Watson, R. (2010) *Migrating Applications to the Cloud: Rehost, Refactor, Revise, Rebuild, or Replace?*". Available by subscription at https://www.gartner.com/en/documents/1485116/migrating-applications-to-the-cloud-rehost-refactor-revi

[3] – Bergey, J. and al. (1999) *Option Analysis for Reengineering (AOR) – Issues and Conceptual Approach.* Available at https://apps.dtic.mil/dtic/tr/fulltext/u2/a370600.pdf

[4] – Barbero, M. (2006) *Architecting the IT Organization – On IT Architecture Tools and Techniques and Their Application To (IT) Organizational Design*. In BearingPoint Financial Services Technical Journal, pp 19-46. Available at https://tinyurl.com/ua4539p

[5] – Dalio, R. *Principles* (2019). Available in print and at https://www.principles.com . The references in this article use the numbering scheme used in printed version of the book[45].

- *Embrace Reality and Deal with It, p. 135*
- *Use the 5-Step Process to Get What You Want Out of Life*, p. 168
- *Spend lavishly on the time and energy you devote to getting in sync, because it's the best investment you can make*, p. 360
- *Keep in mind both the rate of change and the levels of things and the relationships between them*, p. 244
- *Be Imprecise. Understand the concept of 'by-and-large' and approximations*, p. 244
- *Remember that decisions need to be made at the appropriate level, but should be consistent across levels, p. 247*
- *Logic, reason, and common sense are your best tools for synthesizing reality and understanding what to do about it, p 251*
- Find the most believable people possible who disagree with you and understand their reasoning, p. 375
- *Manage as someone operating a machine to achieve a goal*, p. 448
- *Look down on your machine and yourself within it from the higher level*, p. 450
- *Remember that for every case you deal with, your approach should have two purposes: a) move you closer to the goal and b) to train and test your machine (i.e. your people and your design)*, p. 453
- *Diagnose Problems to Get at Their Root Cause*, p. 483

[6] – Hubbard, D. (2014) *How to Measure Anything*

[7] – Barbero M. (2005) *Monte Carlo Analysis for Technology Project Risk Modeling*. In BearingPoint Financial Services Technical Journal, pp 41 – 45. Available at https://tinyurl.com/tz8dckz

[8] – Erder, M.; Pureur P. (2003) *QFD in the Architecture Development Process*. In IT Pro available at https://tinyurl.com/v5383nd

[9] – Pureur, P. (2005) *Using Real Option to Mitigate Risks in Large Projects*. In BearingPoint Financial Services Technical Journal, pp 44 – 52. Available at https://tinyurl.com/tz8dckz

[10] – Martorelli, B.; Nelson L. (2018) *Forrester's Overview Of 20 Cloud Migration Technology Providers*. Available by subscription at https://www.forrester.com/report/Now+Tech+Cloud+Migration+Technology+Q3+2018/-/E-RES143065

[11] – OMG (2016) *Automated Source Code Performance Efficiency*. Available at https://www.omg.org/spec/ASCPEM/1.0/PDF

[12] – OMG (2016) *Automated Source Code Security Measure*. Available at https://www.omg.org/spec/ASCSM/1.0/PDF

[13] – QA Vector (2019*) Cloud migration 2.0: shifting priorities for application modernization in 2019*. Available at https://tinyurl.com/rt2wrp8

Referenced Links and Wikipedia Articles

- On Code Quality Standards – https://www.it-cisq.org/standards/code-quality-standards
- On Software Quality – https://en.wikipedia.org/wiki/Software_quality provides an overview of the topic too.
- On Application Cloud Readiness Tool - https://www.castsoftware.com/products/highlight/cloud-readiness-paas-migration
- On Application Portfolio Management Glossary – https://www.gartner.com/en/information-technology/glossary/application-portfolio-analysis
- On Migration to Cloud – https://www.techrepublic.com/article/73-of-cloud-migrations-take-a-year-or-longer-report-says/
- On Software Modernization – https://en.wikipedia.org/wiki/Software_modernization
- On Application Portfolio Management – https://en.wikipedia.org/wiki/Application_portfolio_management

*** ***

[45] A world of caution: [5] https://tinyurl.com/tz8dckz is best understood as a system of thoughts: none of them should not be applied slavishly for "5.5 Logic, reason, and common sense are your best tools for synthesizing reality and understanding what to do about it", p.251